



Complete Engineering Specification & System Documentation

Autonomous Consumer Protection Agent • Return Window Tracking • CPSC Safety Recall Shield • AI-Drafted Claims

Lead Engineer & Architect:

Agent Framework:

Backend Stack:

Frontend Stack:

Target Domain:

Document Classification:

★ LEAD ENGINEER SYSTEM VISION — V.A. Sai Venkatesh

"Every year, consumers lose hundreds of dollars in missed return windows or unknowingly keep hazardous recalled items in their homes. Aegis changes this: it is a fully autonomous Purchase Guardian that remains 100% silent for routine safe items, and only surfaces when human action is strictly required — with a pre-drafted, 1-click claim ready to send."

System Architecture & Computer Science Foundations

Aegis is organized as a 4-tier distributed system with strict boundary separation between Presentation, API Routing, Agent Orchestration, and Data/External API layers. Every architectural decision is grounded in a formal computer science pattern with explicit engineering rationale.

CS Pattern	Implementation & File Location	Engineering Rationale
Multi-Agent Orchestration	app/agents/ (Strands Agents SDK)	IngestionAgent, TriageAgent, RecallCheckAgent, and DraftAgent are isolated, testable components.
Factory Pattern (ReceiptParserFactory)	app/agents/receipt_factory.py	Dispatches unstructured plaintext, email receipts, or structured JSON to dedicated parsers (TriageAgent, RecallCheckAgent).
Repository Pattern (ItemRepository)	app/repository.py	Wraps all SQLite ORM operations. Zero raw SQL exists outside this class, guaranteeing separation of concerns.
Observer Pattern (EscalationPublisher)	app/publisher.py	Decouples escalation generation from delivery channels. Notifies InAppFeedNotifier and TelegramNotifier.
Singleton Pattern	app/config.py (DatabaseSingleton & BedrockConfigSingleton)	The BedrockConfigSingleton engine prevents file locking; shared Bedrock config manages AWS credentials and model endpoints.
Strategy Pattern (TriageAgent)	app/agents/triage_agent.py	Defines interchangeable triage rules (ReturnWindowRule, WarrantyExpiringRule, RecallMatchRule).

Core Data Structures

Data Structure	Implementation	Algorithmic Justification
Min-Heap (DeadlineHeap)	app/data_structures.py Python heapq	In-memory priority queue keyed by nearest deadline timestamp min(return_deadline, warranty_deadline).
LRU Cache (LRUProductCache)	app/data_structures.py collections.OrderedDict	Caches CPSC recall query responses per product name and model to avoid redundant network round-trips.

Part I.1: Autonomous Multi-Agent Architecture Topology

★ 4-Agent Pipeline Topology — V.A. Sai Venkatesh (Lead Engineer)

Each API request flows through a deterministic four-agent pipeline. The IngestionAgent parses unstructured receipt data, the TriageAgent applies Strategy-pattern rules to decide whether escalation is warranted, the RecallCheckAgent cross-references federal CPSC data with LRU caching, and the DraftAgent uses Amazon Bedrock to pre-fill ready-to-send return or recall claim messages.

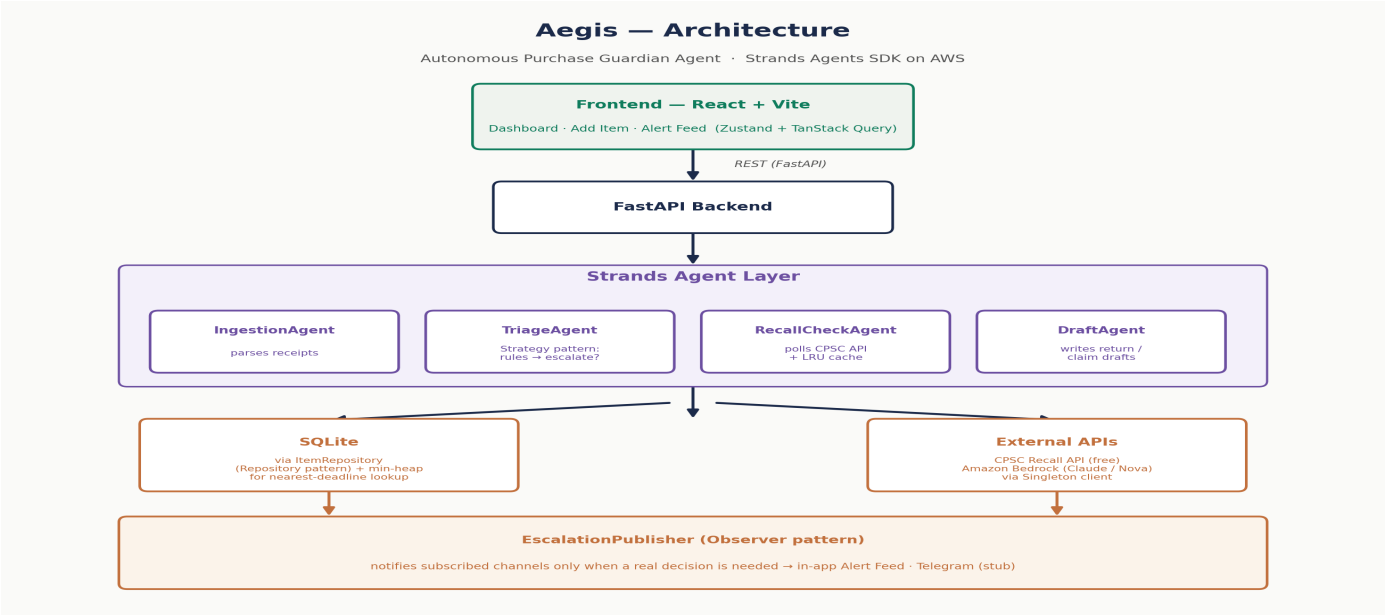


Figure 1: Aegis Multi-Agent Architecture — 4-Tier Distributed System

Why It Is Needed:

Consumer products generate unstructured receipt data across dozens of merchant formats. A single monolithic parser would be brittle and untestable. By isolating ingestion, triage, recall checking, and claim drafting into specialized agents, each component can be tested and replaced independently without affecting the rest of the pipeline.

Operational Procedures & Workflow:

1. Submit a receipt (plaintext, JSON, or image OCR) via POST /items or the Track Receipt UI.
2. IngestionAgent extracts item name, merchant, purchase date, price, return window, and warranty period.
3. ItemRepository persists the item to SQLite and atomically updates the Min-Heap priority queue.
4. TriageAgent runs all Strategy rules concurrently; if warranted, DraftAgent generates a claim draft.
5. EscalationPublisher broadcasts to InAppFeedNotifier and TelegramNotifier simultaneously.

Part I.2: Application Dashboard & Guardian Feed

★ Silent Tracking vs. Actionable Escalation Principle — V.A. Sai Venkatesh

The Guardian Feed is the primary consumer interface. Items are displayed in Min-Heap priority order (nearest deadline first). Routine items within safe return windows and with zero CPSC recall matches display a ■ Silent & Clear badge and do not trigger any notifications. Only two conditions trigger escalation: a return window closing within 5 days, or a live CPSC recall match.

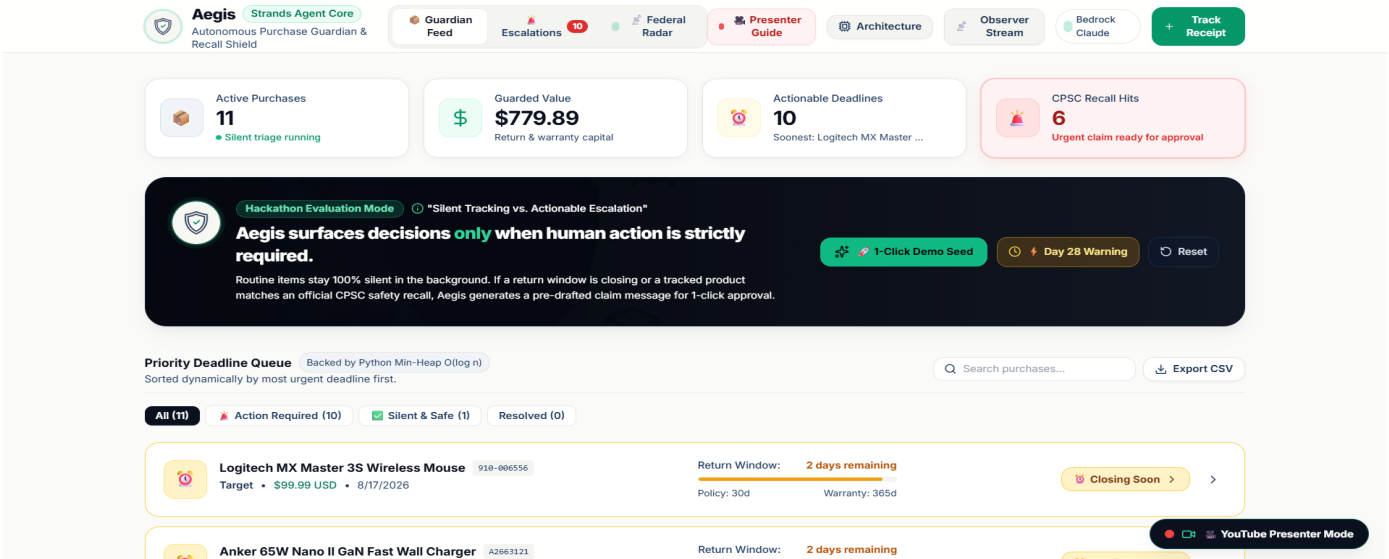


Figure 2: Guardian Feed Dashboard — Min-Heap Priority Queue with Silent vs. Escalation States

Why It Is Needed:

Traditional purchase-tracking apps create alert fatigue by bombarding users with daily reminders. Aegis inverts this paradigm: it is strictly autonomous and silent until a real human decision point is reached. This design preserves user attention for moments that genuinely matter.

Operational Procedures & Workflow:

1. Open the dashboard at <http://localhost:5173>.
2. Click ■ "1-Click Demo Seed" in the evaluation banner to load four sample receipts.
3. Observe routine items (Anker Charger, Nike Shoes) display ■ Silent & Clear status.
4. Observe the Logitech Mouse (Day 28) and Cade Toys (CPSC recall) display urgent escalation states.
5. Use the Priority tabs: All | Action Required | Silent & Safe | Resolved.
6. Search or export the full purchase registry as CSV for audit.

Part I.3: Escalations & 1-Click Bedrock Claim Drafting

★ DraftAgent — Amazon Bedrock Multi-Strategy Claim Generation — V.A. Sai Venkatesh

When the TriageAgent determines that human action is required, the DraftAgent invokes Amazon Bedrock (Claude 3 Haiku via Strands) to generate three ready-to-send claim drafts simultaneously: a Merchant Return email, an Official CPSC Recall Claim, and a Credit Card Purchase Protection request. All three are surfaced in the Escalations tab for 1-click approval or dismissal.

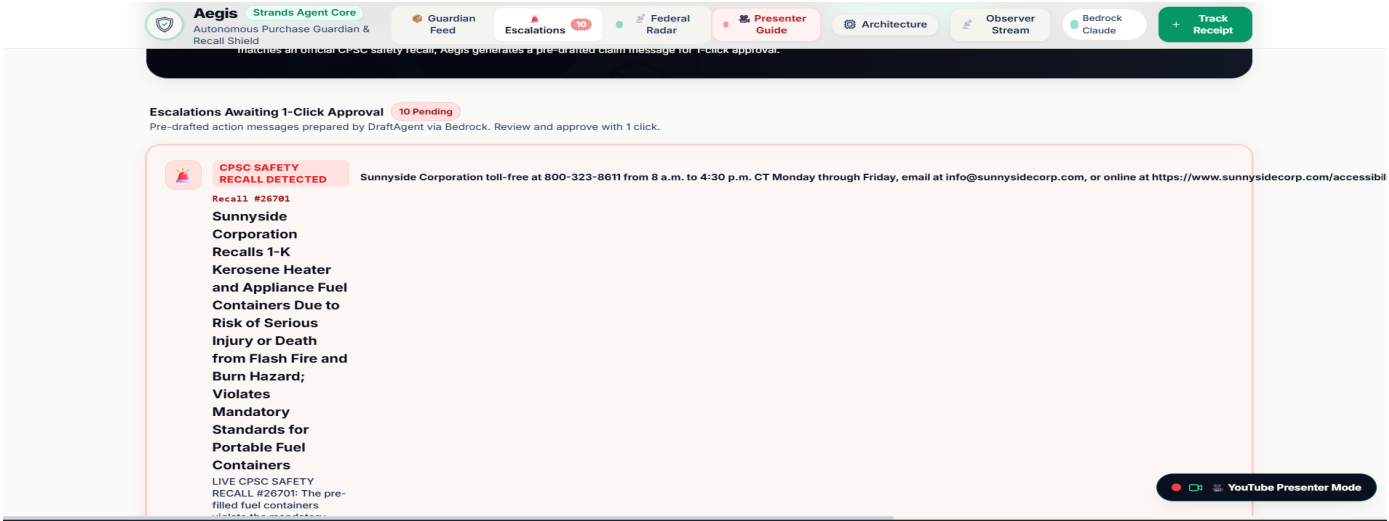


Figure 3: Escalations Tab — CPSC Safety Recall with Pre-Drafted Claim (Pending Approval)

Escalation Type Matrix:

Escalation Type	Trigger Condition	DraftAgent Output	Severity
return_window	Return deadline $\leq$ 5 days remaining	Merchant RMA return label email pre-filled with item details	High
recall_match	CPSC API returns recall match for item	Official CPSC Recall Remedy claim + Credit Card Protection request	Critical
warranty_expiring	Warranty deadline $\leq$ 30 days remaining	Manufacturer warranty claim reminder with purchase details	Medium

Operational Procedures & Workflow:

1. Navigate to the Escalations tab in the top navigation bar.
2. Review the pre-drafted claim prepared by DraftAgent with the official CPSC recall identifier.
3. Switch between claim strategy tabs: Merchant Return | CPSC Claim | Card Protection.
4. Click “1-Click Approve & Dispatch” to mark the escalation as approved.
5. An in-app toast confirms the Observer pattern notification has been dispatched.
6. The item status transitions to returned or claimed in the database and Min-Heap.

Part I.4: Live Federal CPSC Safety Recall Radar

★ Real-Time saferproducts.gov API Integration — V.A. Sai Venkatesh

The Federal Recall Radar queries the U.S. Consumer Product Safety Commission (CPSC) SaferProducts REST API in real time — this is NOT static mock data. Search results include the official recall ID, hazard description, remedy instructions, and consumer contact information. Clicking “Shield Product” on any result immediately ingests it into Aegis and generates an emergency recall escalation with a pre-drafted claim.

The screenshot displays the 'Federal Recall Radar' interface within the Aegis application. The top navigation bar includes 'Aegis Strands Agent Core', 'Guardian Feed', 'Escalations 10', 'Federal Radar', 'Presenter Guide', 'Architecture', 'Observer Stream', 'Bedrock Claude', and a 'Track Receipt' button. The main section is titled 'Federal Recall Radar — Live CPSC Database' with a 'saferproducts.gov LIVE' badge. It features a search bar with 'Heater' entered and a 'Scan Registry' button. Below the search bar are 'Quick Scans' for various product categories: Heater, Charger, Peloton, Bicycle, Stroller, Ember, Battery, Generator, Helmet, and Toy. Two recall cards are shown:

- CPSC #26701 (8/13/2026):** Sunnyside Corporation Recalls 1-K Kerosene Heater and Appliance Fuel Containers Due to Risk of Serious Injury or Death from Flash Fire and Burn Hazard; Violates Mandatory Standards for Portable Fuel Containers. The hazard states that pre-filled fuel containers violate mandatory safety standards, posing a deadly risk of flash fire and burn hazard. The remedy instructs consumers to stop using the recalled fuel bottles immediately, place them out of reach of children, and contact Sunnyside for a full refund. Consumers will be asked to write "RECALLED" with permanent marker on the bottle and submit a photo of it to info@sunnysidecorp.com. A note mentions that Kerosene fuel is hazardous and should be disposed of or recycled by following local hazardous waste procedures.
- CPSC #26532 (6/4/2026):** Vornado Air Recalls SRTH Small Room Tower Heaters Due to Fire Hazard. The hazard states that the recalled heater's fan blade can detach from the motor shaft, which can cause the fan to slow or stop, leading to overheating and melting of the enclosure and internal parts. Melted internal parts can ignite and breach the enclosure if the thermal cut off or fuse does not timely activate, posing a fire hazard. The remedy instructs consumers to stop using the recalled heaters immediately and contact Vornado for instructions on how to submit photos of the product and proof of destruction to receive a full refund.

Additional text at the bottom of the first card mentions that this recall involves Sunnyside gallon 1-K Kerosene Heater and Appliance Fluid Portable Fuel Containers with batch number 26082. The bottles come in white plastic containers with white caps and have a multicolor label with the words "sunnyside" and "kerosene". The container has...

A 'YouTube Presenter Mode' button is visible in the bottom right corner of the interface.

Figure 4: Federal Recall Radar — Live saferproducts.gov API Query for "Heater"

Why It Is Needed:

The CPSC announces thousands of product recalls annually covering fire hazards, choking hazards, toxic chemical exposure, and battery ingestion risks. Most consumers never check these databases. Aegis gives consumers a direct window into live federal recall data for any product keyword, enabling immediate protective action.

Operational Procedures & Workflow:

1. Click “Federal Radar” in the top navigation bar.
2. Type any product keyword (heaters, chargers, toys, batteries) or click a Quick Scan chip.
3. Click “Scan Registry” to query the live CPSC SaferProducts API.
4. Review official recall cards showing recall ID, hazard, remedy, and consumer contact.
5. Click “Shield Product” on any match to instantly track it and generate an emergency escalation.

Part I.5: Receipt Ingestion & ReceiptParserFactory

★ ReceiptParserFactory (Factory Pattern) — V.A. Sai Venkatesh

The Track Receipt modal accepts three input modes: Plain Text (paste any receipt or e-commerce email), JSON (structured product payload), and Image Snapshot (in-browser Tesseract.js OCR). The ReceiptParserFactory dispatches the input to the appropriate parser without exposing parsing internals to the caller. The IngestionAgent then standardizes extracted data and maps return policies across 16+ major retailers automatically.

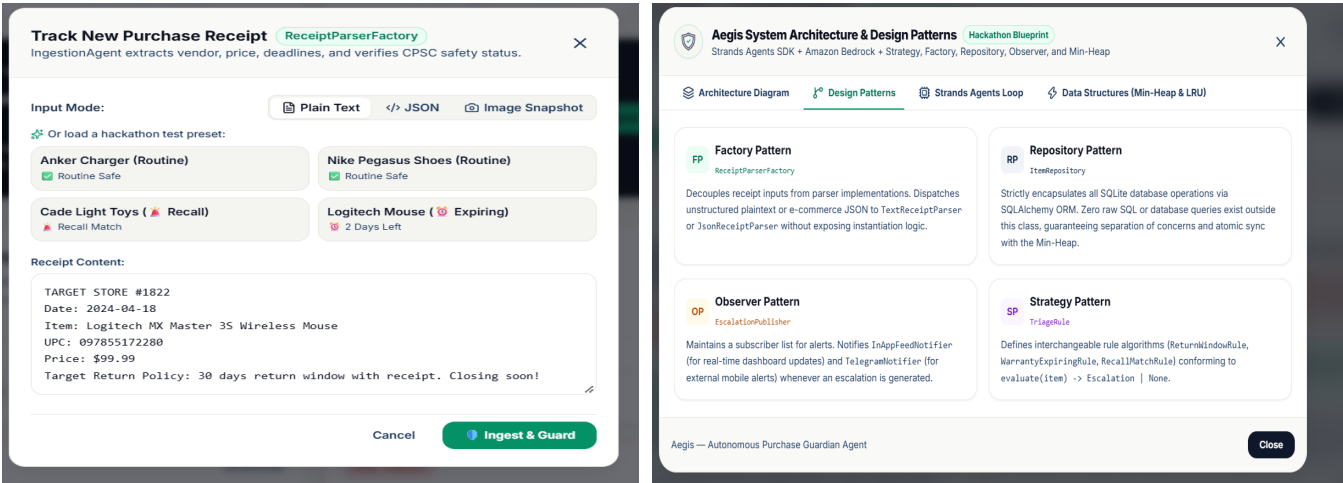


Figure 5 (left): Track Receipt Modal — ReceiptParserFactory input modes. Figure 6 (right): Architecture Modal — Design Patterns tab.

Retail Policy Mapping (ReceiptParserFactory knowledge base):

Merchant	Return Window	Warranty Support	Notes
Amazon	30 days	Varies by category	Extended for Prime members
Target	30 days (90 for RedCard)	Manufacturer	Electronics: 30 days
Best Buy	15 days (30 for Elite)	Geek Squad Protection	Open-box: 15 days
Walmart	30 days	Manufacturer	Electronics: 15 days
Costco	90 days (electronics)	Varies	Lifetime for some categories
Apple	14 days	1 year AppleCare base	Extended AppleCare+ available

Part II: Complete REST API Specification

Core Production Endpoints

Method & Path	Subsystem	Description & Operation
POST /items	IngestionAgent	Ingests receipt text or JSON. Runs IngestionAgent parse, ItemRepository persist, TriageAgent evaluation, and Bedrock confirmation.
GET /items	Repository	Returns all tracked purchases sorted by nearest deadline (Min-Heap prioritized). Supports ?status= filter.
GET /items/{id}	Repository	Returns full purchase record including escalations, days remaining, and CPSC certificate status.
DELETE /items/{id}	Repository	Removes item from SQLite and synchronizes Min-Heap.
GET /alerts	Escalation	Lists pending escalations requiring user action. Frontend polls this endpoint every 30 seconds.
POST /alerts/{id}/approve	Observer	1-click approval. Marks escalation approved, transitions item status, and triggers Observer notifications.
POST /alerts/{id}/dismiss	Observer	Dismisses an escalation without taking action.
GET /recalls/live-search?q=	RecallCheckAgent	Queries live U.S. CPSC SaferProducts API in real time. Returns recall cards with hazard, remedy, and consumer instructions.
POST /recalls/protect	RecallCheckAgent	Shields a recalled product from live CPSC results into Aegis. Generates emergency escalation and Bedrock clarification.
POST /demo/seed	Seed Service	Loads 4 evaluation receipts: 2 silent routine items, 1 safety recall, 1 expiring return window.
POST /demo/simulate-expiry	TriageAgent	Time-travel: fast-forwards an item to Day 28 to trigger urgent return deadline escalation for live demos.
POST /demo/reset	Repository	Clears all items and escalations for fresh evaluation runs.
GET /stats	System	Guardian telemetry: active purchase count, guarded value, actionable deadline count, and Amazon Bedrock confirmation rate.
GET /health	System	Deployment readiness probe. Returns service name, Bedrock status, and Min-Heap size.
GET /events	Observer	Returns recent in-app events and Telegram dispatch log from the EscalationPublisher Observer pattern.

HTTP Status Code & Resilience Dictionary

Status Code	Condition & Cause	Platform Resolution & Handling
200 OK	Successful pipeline analysis, ingestion, and confirmation.	Returns structured JSON payload with full item/escalation schema.
201 Created	New item successfully ingested and persisted.	ItemRepository returns ItemResponse including computed deadline and recall status.
204 No Content	Item successfully deleted	Min-Heap synchronized. No response body.
404 Not Found	Item or escalation ID does not exist	FastAPI returns descriptive JSON error message.
422 Unprocessable Entity	Invalid receipt input or malformed JSON payload	Pydantic v2 validation error with field-level detail.
500 Server Error	Missing AWS credentials or invalid Bedrock configuration	Bedrock confirmation fails back to heuristic local mode with identical response schema.

Part III: Database Schema, Deployment & Quickstart

SQLite Schema: Items & Escalations Tables

Field Name	Data Type	Description & Role	Index Rule
id	UUID string	Primary item identifier	Primary key
name	string	Product name from receipt	Text search
merchant	string	Retailer or vendor name	Filter index
price	float	Purchase price in currency units	Aggregation
purchase_date	datetime UTC	Date of purchase (normalized to UTC)	Sort ascending
return_window_days	integer	Merchant return policy duration in days	Computed field
return_deadline	datetime UTC	purchase_date + return_window_days	Min-Heap key
warranty_days	integer	Manufacturer warranty duration in days	Computed field
warranty_deadline	datetime UTC	purchase_date + warranty_days	Min-Heap key
status	enum string	active returned claimed dismissed	Filter index
model_number	string (nullable)	Product model for CPSC recall cross-reference	LRU cache key

Deployment Architecture

Option	Platform	Configuration
Option A — Local Dev	Python 3.10+ / Node 18+	Backend: cd backend && python run.py (port 8000) Frontend: cd frontend && npm run dev (port 5173)
Option B — Docker	Docker Compose	Multi-stage Dockerfile. AWS credentials mounted as environment variables. CORS pre-configured for containerized frontend URL.
Option C — Cloud	AWS / Fly.io / Railway	FastAPI ASGI via uvicorn. AWS_REGION, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, BEDROCK_MODEL_ID environ

Quickstart Guide

- Prerequisites:** Python 3.10+ and Node.js 18+.
- Backend:** cd backend → pip install -r requirements.txt → python run.py
- Unit Tests:** \$env:PYTHONPATH="backend"; python -m pytest backend/tests/test_backend.py -v (5 tests pass)
- Frontend:** cd frontend → npm install → npm run dev
- Open:** http://localhost:5173 in your browser.
- Evaluate:** Click "1-Click Demo Seed", observe silent vs. escalated item states, then navigate Escalations.
- Optional Bedrock:** Create backend/.env with AWS_REGION, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, BEDROCK_MODEL_ID.

★ ARCHITECTURAL SIGN-OFF — V.A. Sai Venkatesh (Lead Engineer)

"Aegis represents a complete, deterministic engineering solution for autonomous consumer protection. By fusing Strands Agents SDK multi-agent orchestration, Amazon Bedrock LLM claim drafting, live CPSC federal recall API integration, Min-Heap deadline tracking, and Observer-pattern multi-channel notifications, Aegis delivers silent background monitoring for routine items and precision escalation only when human action is strictly required."